SOMAIYA VIDYAVIHAR UNIVERSITY
K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)

Somaiya TRUST

Module 3
Informed Search

Nirmala Shinde Baloorkar
Assistant Professor
Department of Computer Engineering

Outline

- Best-first search
  - Greedy best-first search
  - A^* search
- Heuristics
- Local search algorithms
  - Hill-climbing search

Review: Tree search

- \input{\file{algorithms}{tree-search-short-algorithm}}
- A search strategy is defined by picking the order of node expansion

Best-first search

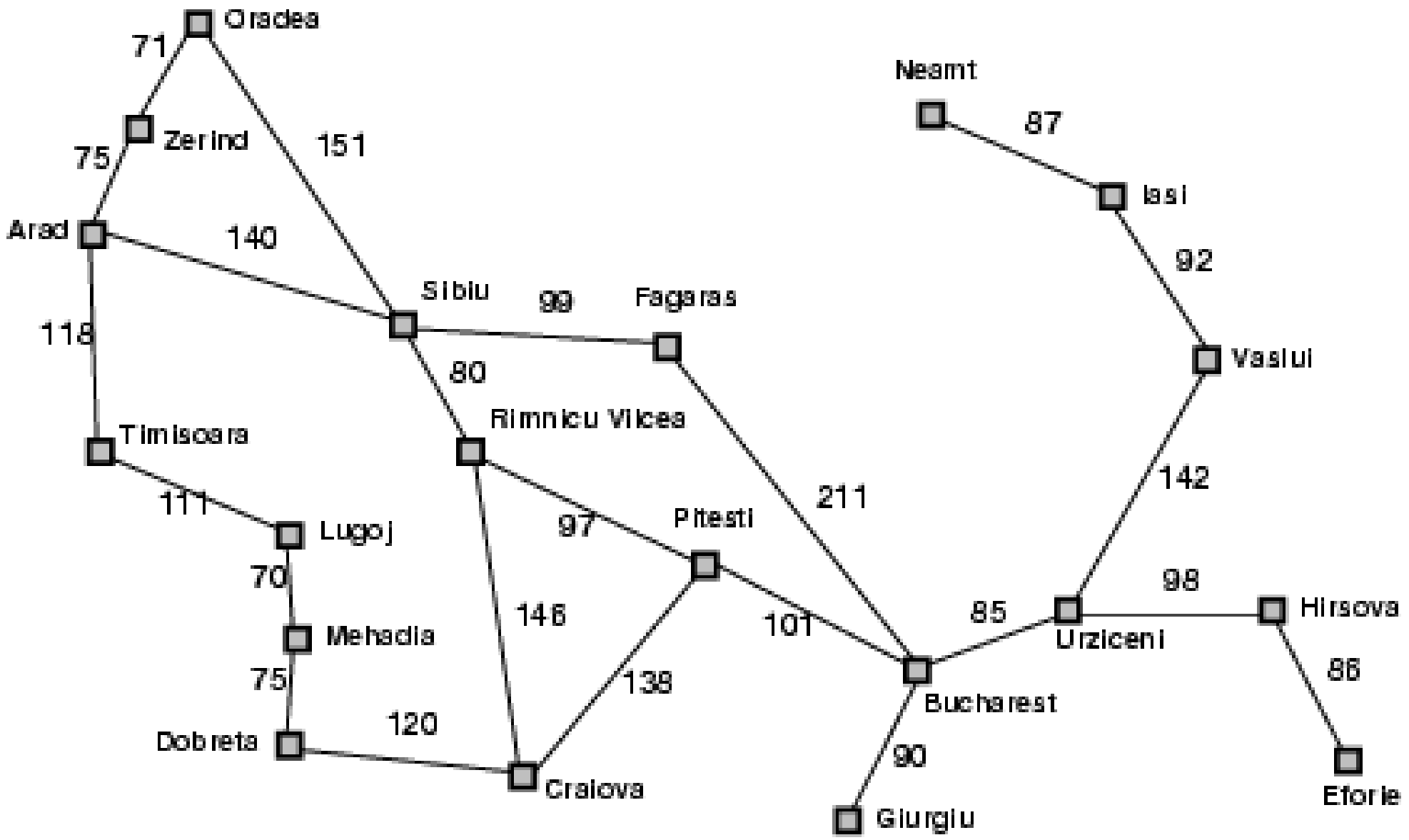
- Idea: use an evaluation function *f(n)* for each node
  - estimate of "desirability“
  - Expand most desirable unexpanded node

- Implementation:

  Order the nodes in fringe in decreasing order of desirability

- Special cases:
  - greedy best-first search
  - A^* search

Romania with step costs in km



Straight–line distance to Bucharest

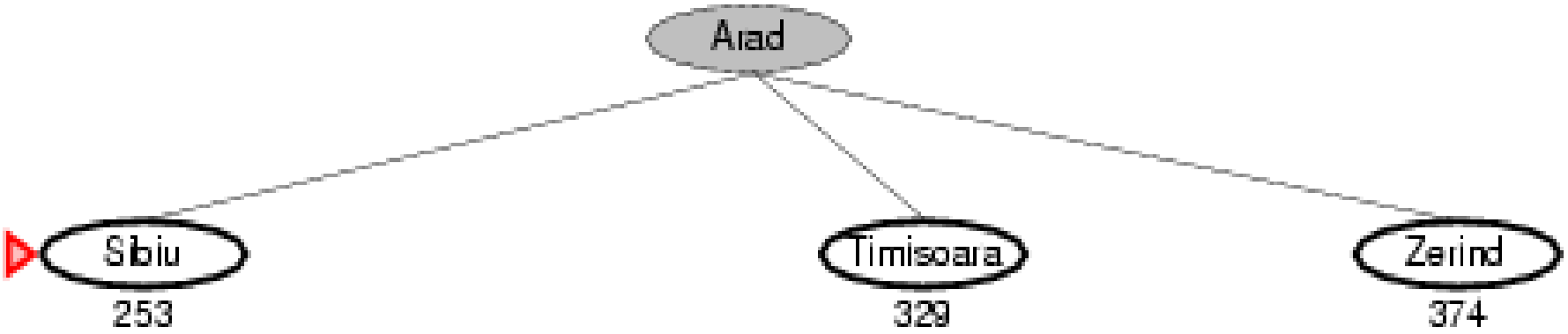
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	176
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	10
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy best-first search

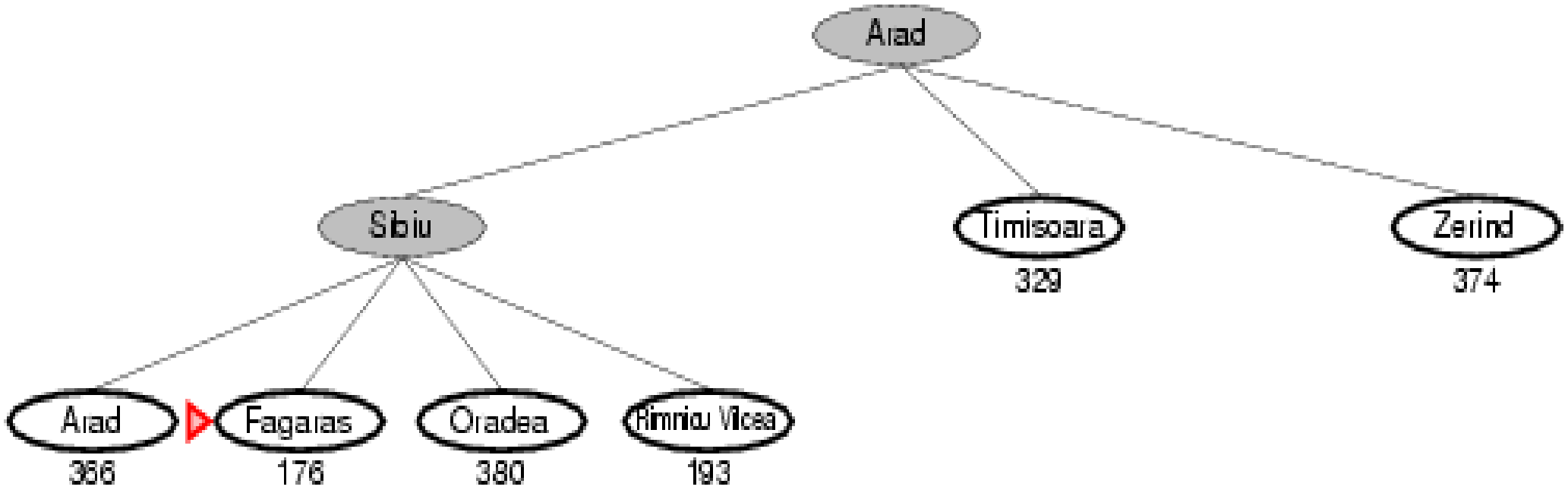
- Evaluation function $f(n) = h(n)$ (heuristic)= estimate of cost from n to *goal*
- e.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest
- Greedy best-first search expands the node that appears to be closest to goal

Greedy best-first search example

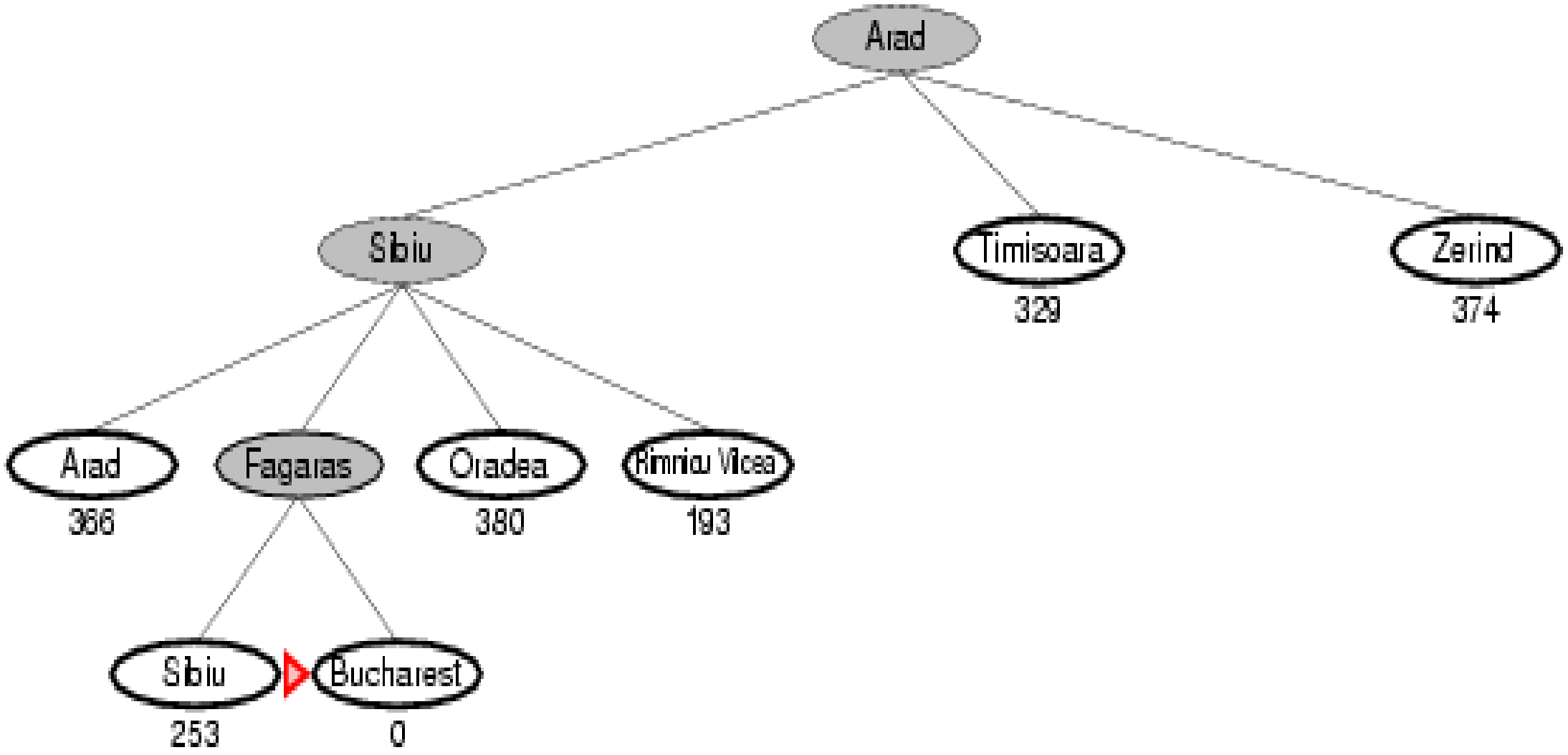
Greedy best-first search example



Greedy best-first search example



Greedy best-first search example



Properties of greedy best-first search

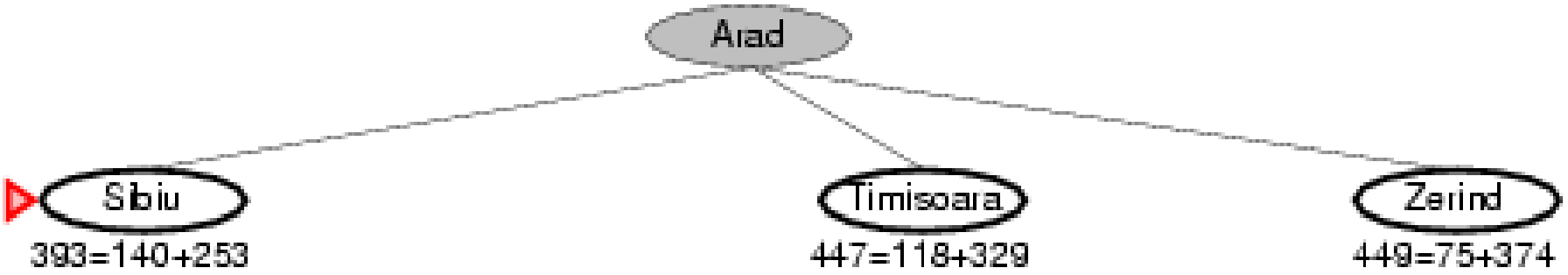
- Complete? No – can get stuck in loops, e.g., Iasi → Neamt → Iasi → Neamt →
- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
- Space? $O(b^m)$ -- keeps all nodes in memory
- Optimal? No

A* search

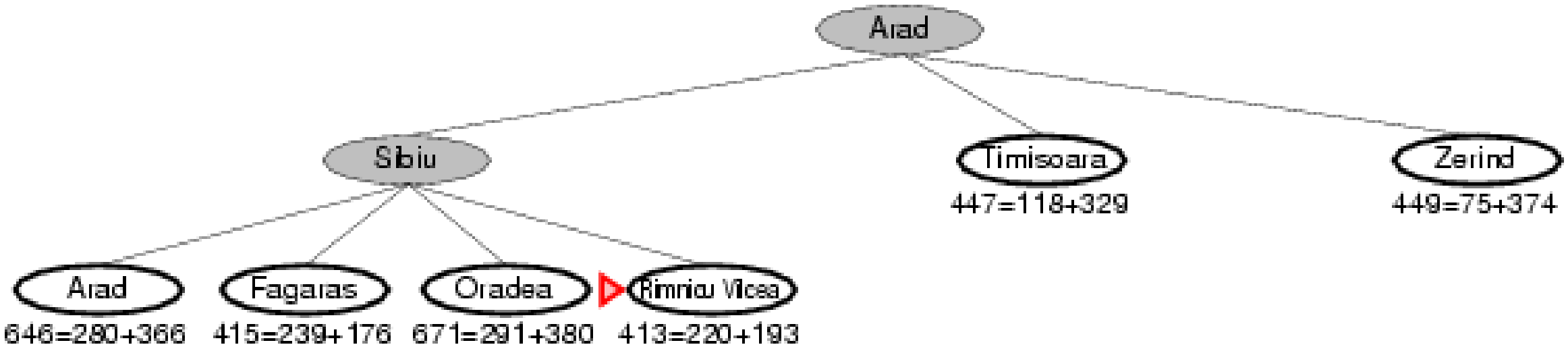
- Idea: avoid expanding paths that are already expensive
- Evaluation function $f(n) = g(n) + h(n)$
- $g(n)$ = cost so far to reach n
- $h(n)$ = estimated cost from n to goal
- $f(n)$ = estimated total cost of path through n to goal

A* :

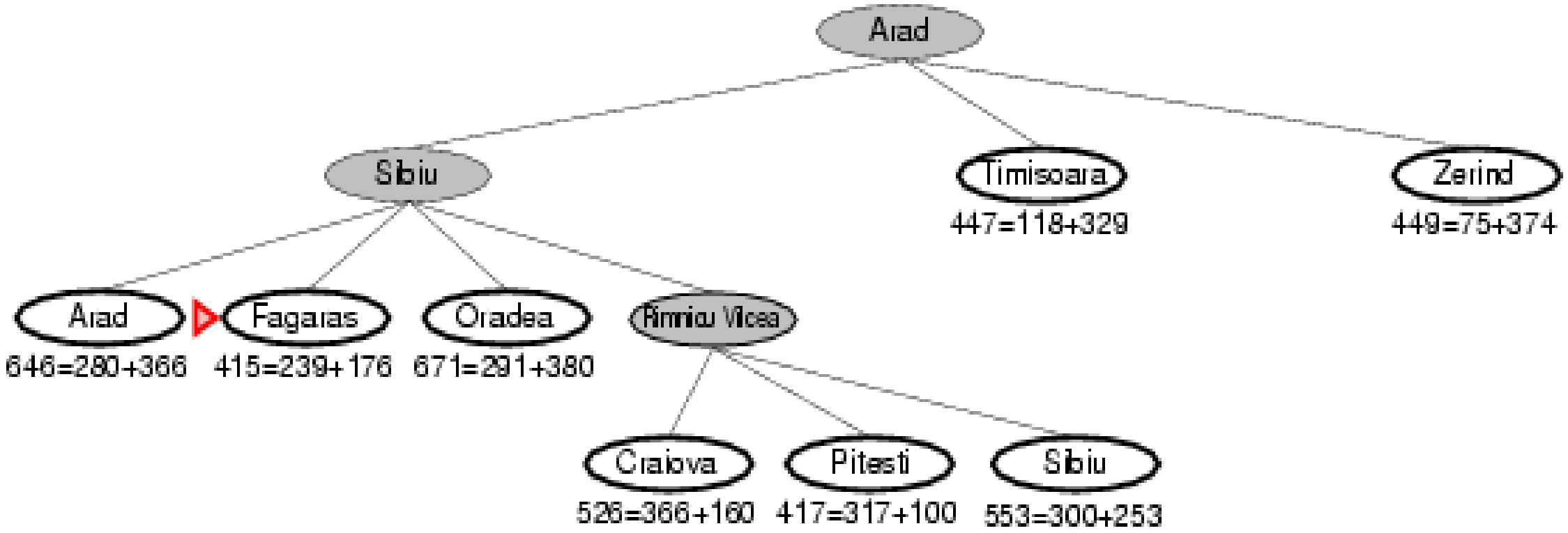
A* search example

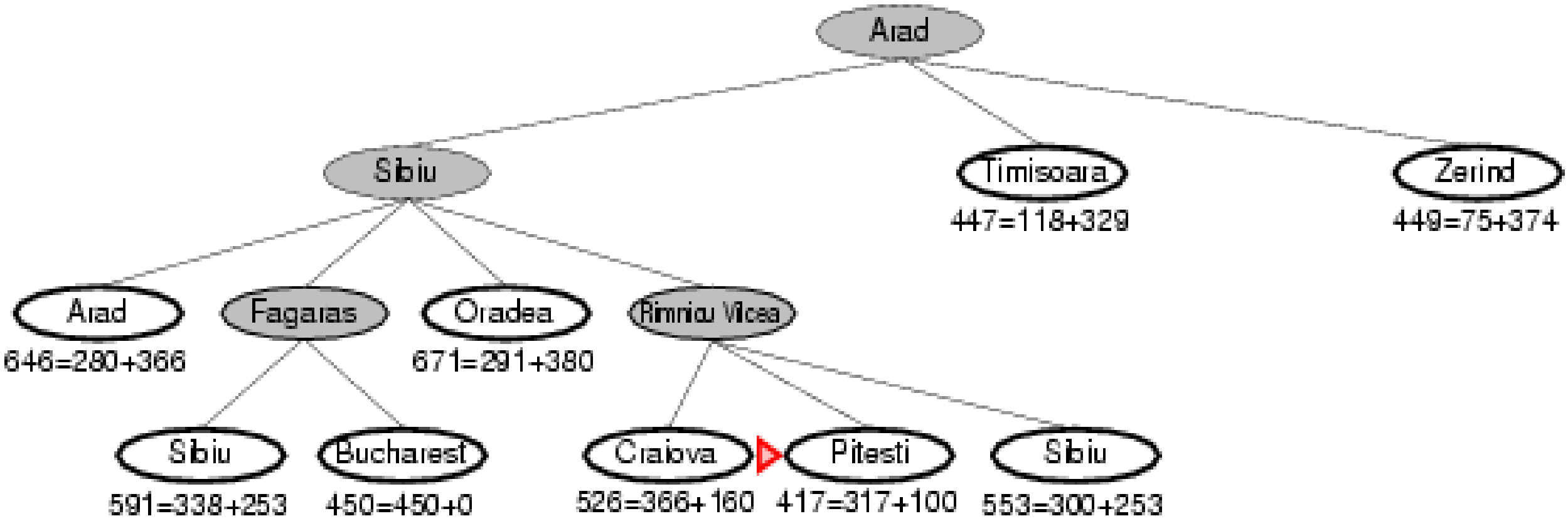


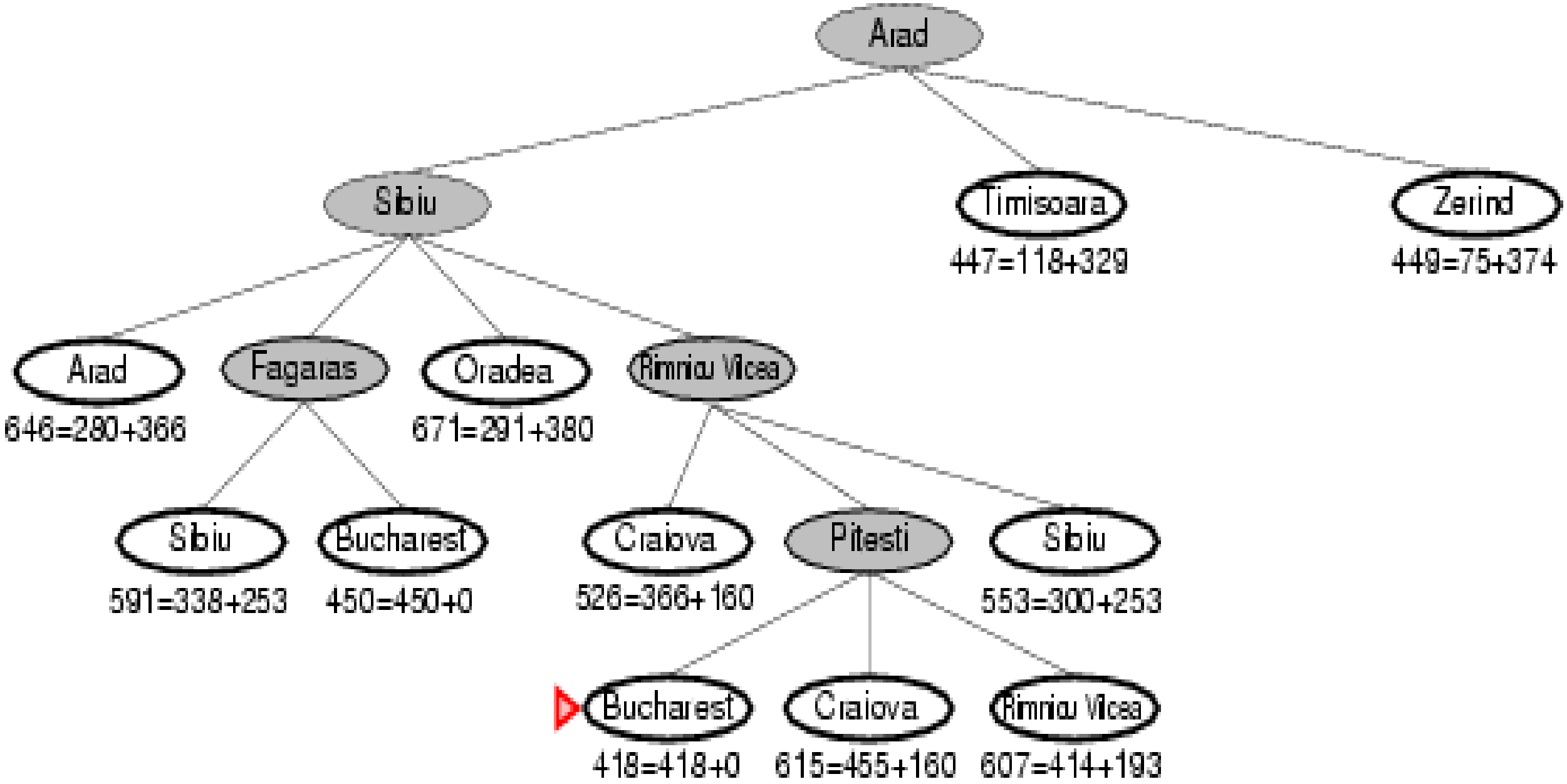
A* search example



A* search example



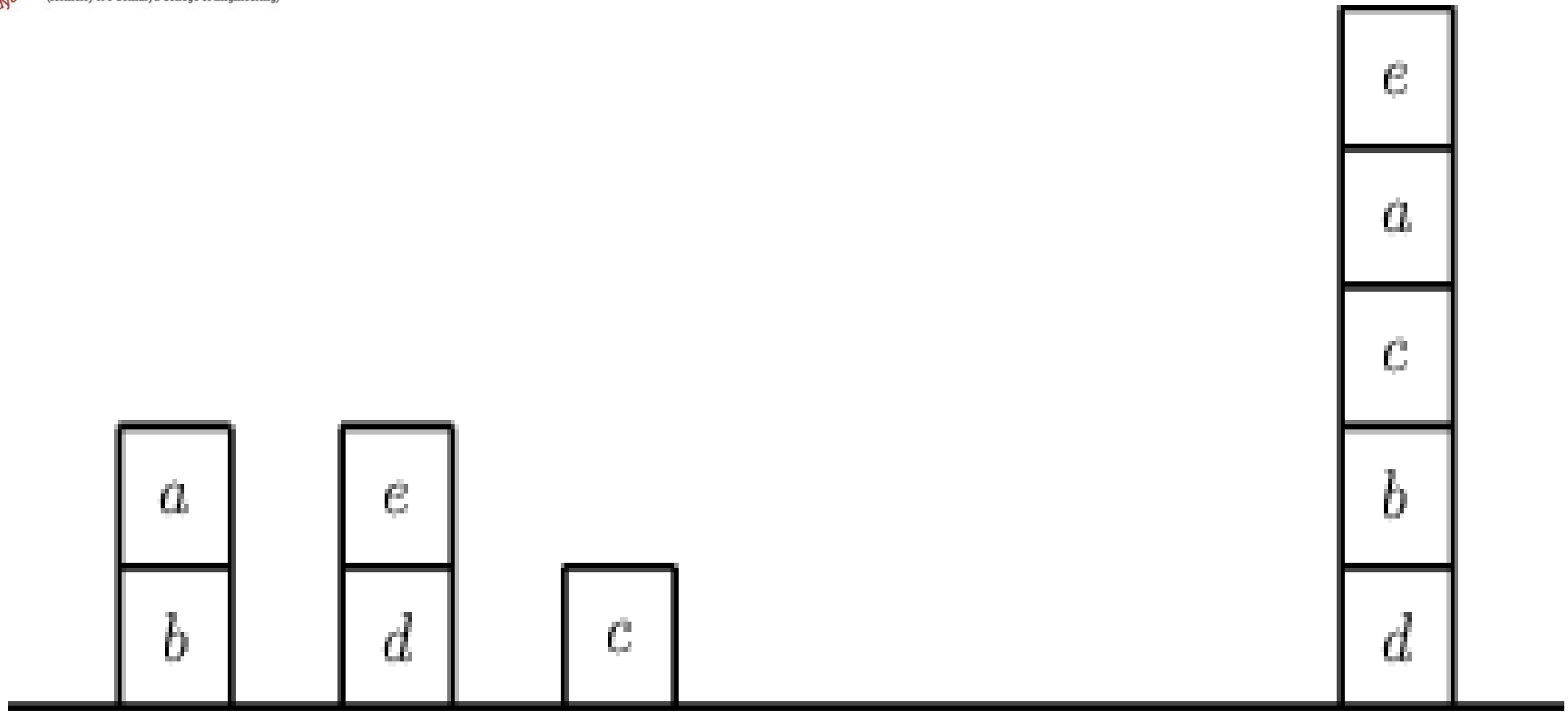

Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
Oradea
671=291+380
Rimnicu Vilcea
Sibiu
591=338+253
Bucharest
450=450+0
Craiova
526=366+160
Pitesti
417=317+100
Sibiu
553=300+253


Arad
Sibiu
Timisoara
447=118+329
Zerind
449=75+374
Arad
646=280+366
Fagaras
Oradea
671=291+380
Rimnicu Vilcea
Sibiu
591=338+253
Bucharest
450=450+0
Craiova
526=366+160
Pitesti
Sibiu
553=300+253
Bucharest
418=418+0
Craiova
615=455+160
Rimnicu Vilcea
607=414+193

Problems and solutions

- 8-puzzle problem
- Block’s world problem
- Graph coloring problem?

- Heuristics?


a
b
e
d
c
e
a
c
b
d

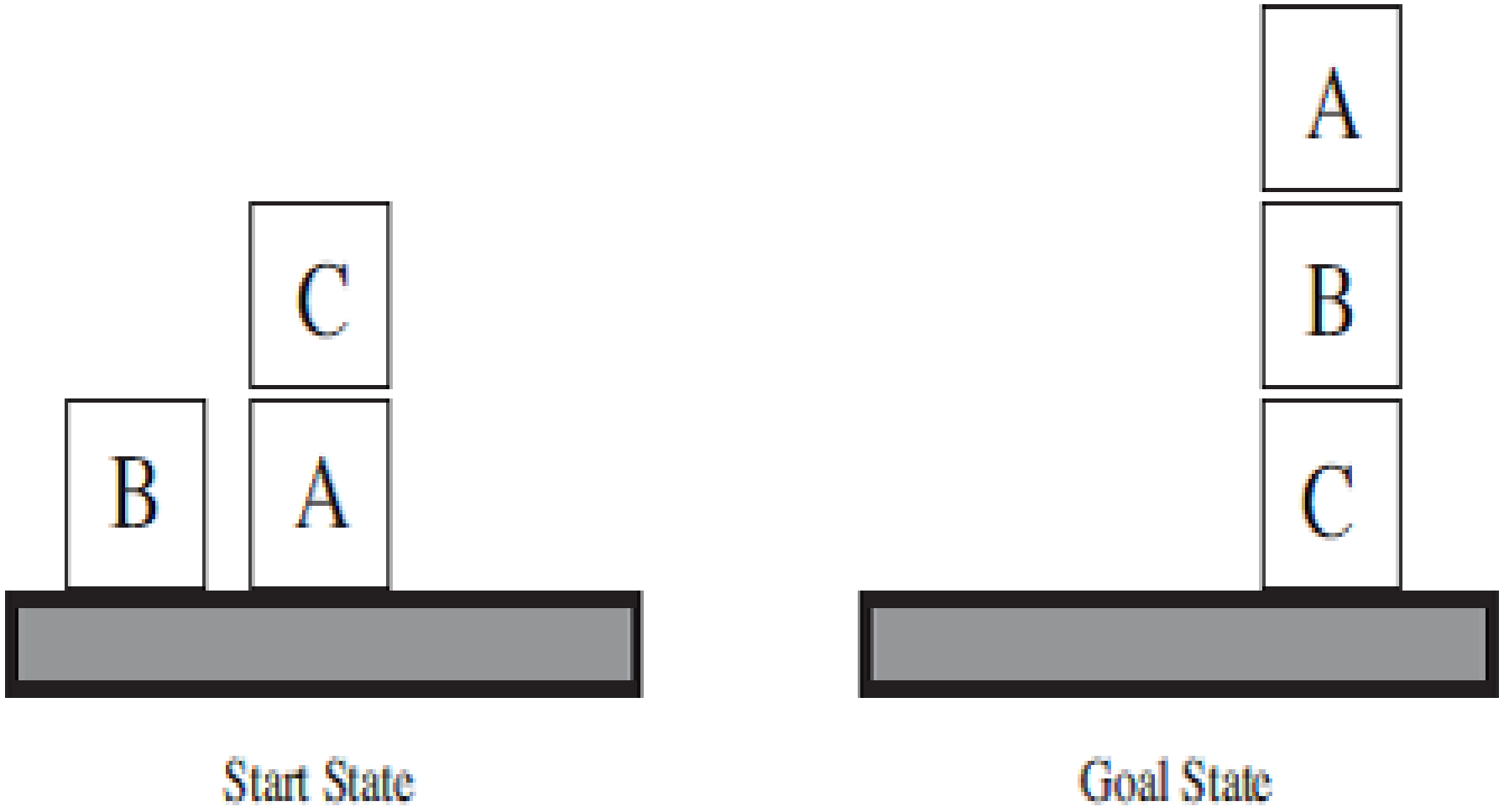


Figure 10.4 Diagram of the blocks-world problem in Figure 10.3.

heuristic

- Definition- Approximation of estimation of cost of current node to goal node.

Admissible heuristics

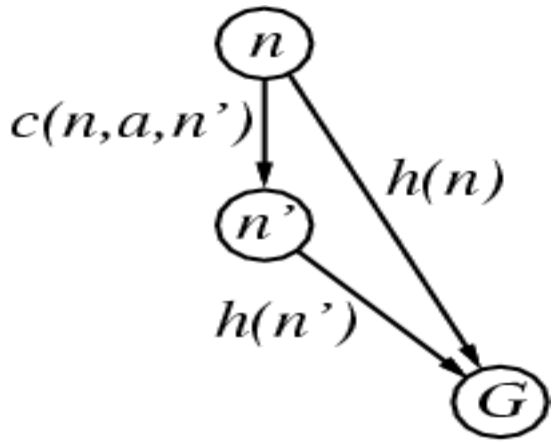
- A heuristic $h(n)$ is admissible if for every node n,

  $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from n.

- An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic
- Example: $h_{SLD}(n)$ (never overestimates the actual road distance)

Consistent heuristics

- A heuristic is consistent if for every node *n*, every successor *n'* of *n* generated by any action *a*,
- $h(n) \leq c(n,a,n') + h(n')$



- If *h* is consistent, we have

$$
\begin{aligned}
f(n') &= g(n') + h(n') \\
&= g(n) + c(n,a,n') + h(n') \\
&\geq g(n) + h(n) \\
&= f(n)
\end{aligned}
$$

- i.e., *f(n)* is non-decreasing along any path.
- If *h(n)* is consistent, A* using `GRAPH-SEARCH` is optimal

Properties of A*

- Complete? Yes (unless there are infinitely many nodes with f ≤ *f(G)*)
- Time? Exponential
- Space? Keeps all nodes in memory
- Optimal? Yes

Admissible heuristics

E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance

(i.e., no. of squares from desired location of each tile)

7	2	4
5		6
8	3	1

Start State

	1	2
3	4	5
6	7	8

Goal State

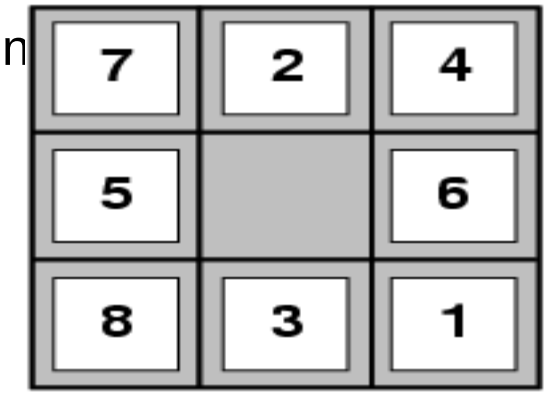
- $h_1(S) = ?$
- $h_2(S) = ?$
-

Admissible heuristics

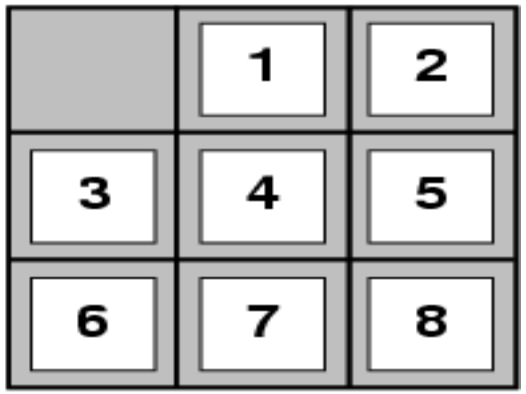
E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance

(i.e., no. of squares from desired location



Start State



Goal State

- $h_1(S)$ = ? 8
- $h_2(S)$ = ? 3+1+2+2+2+3+3+2 = 18

Relaxed problems

- A problem with fewer restrictions on the actions is called a relaxed problem
- The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem
- If the rules of the 8-puzzle are relaxed so that a tile can move anywhere, then $h_1(n)$ gives the shortest solution
- If the rules are relaxed so that a tile can move to any adjacent square, then $h_2(n)$ gives the shortest solution

Informed search strategy

- Uses evaluation function
- Evaluation measures distance from goal state
- One problem can have multiple different evaluation functions
- The key component of evaluation functions is heuristic function.

Heuristics

- Experience-based techniques for problem solving, learning, and discovery
- Generally used where the exhaustive search is impractical.
- Speeds up the process of finding a satisfactory solution via mental shortcuts to ease the cognitive load of making a decision.

Heuristic Function

- In heuristic function, additional knowledge of problem is imparted to the search algorithm.
- Use heuristic function that associates a value to each state.
- This value computed by applying heuristic function on the state.
- This makes one non goal state is more promising than the other one.

Heuristic function… contd

- Heuristic value
- Goal oriented search
- Increases efficiency
- Not optimal always
- Solution in bounded time

Design Of Heuristic Function

- Heuristics are rules of thumb, do not guarantee for a solution to a problem.
- Can be effective if applied correctly, require domain specific information
- Heuristic search function estimates cost from current state to goal, presuming function is efficient.

Comparison

Blind search

- Only have knowledge about already explored nodes
- No knowledge about how far a node is from goal state

Heuristic search

- Estimates “distance” to goal state
- Guides search process toward goal state
- Prefers states that lead close to and not away from goal state

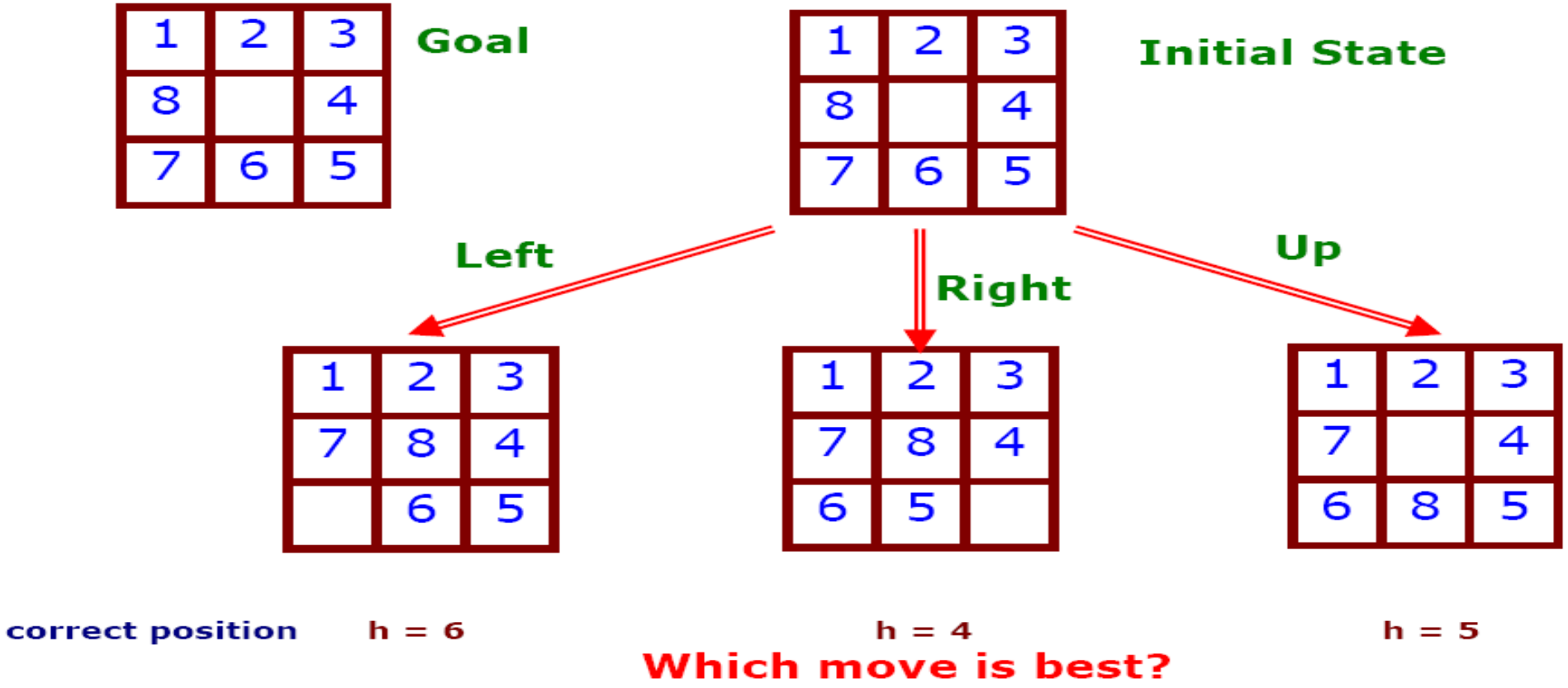
Heuristic function

- 8 Puzzle problem

Heuristic Function:

- Count the number of tiles that are in place.
- H. value for goal state : 8

8 Puzzle problem



8 Puzzle problem

Apply the Heuristic : different approaches

- Count correct position of each tile, compare to goal state
- Count incorrect position of each tile, compare to goal state
- Count how far away each tile is from it’s correct position.

8 Puzzle problem

Approaches	Left	Right	Up
1. Count correct position	6	4	5
2. Count incorrect position	2	4	3
3. Count how far away	2	4	4

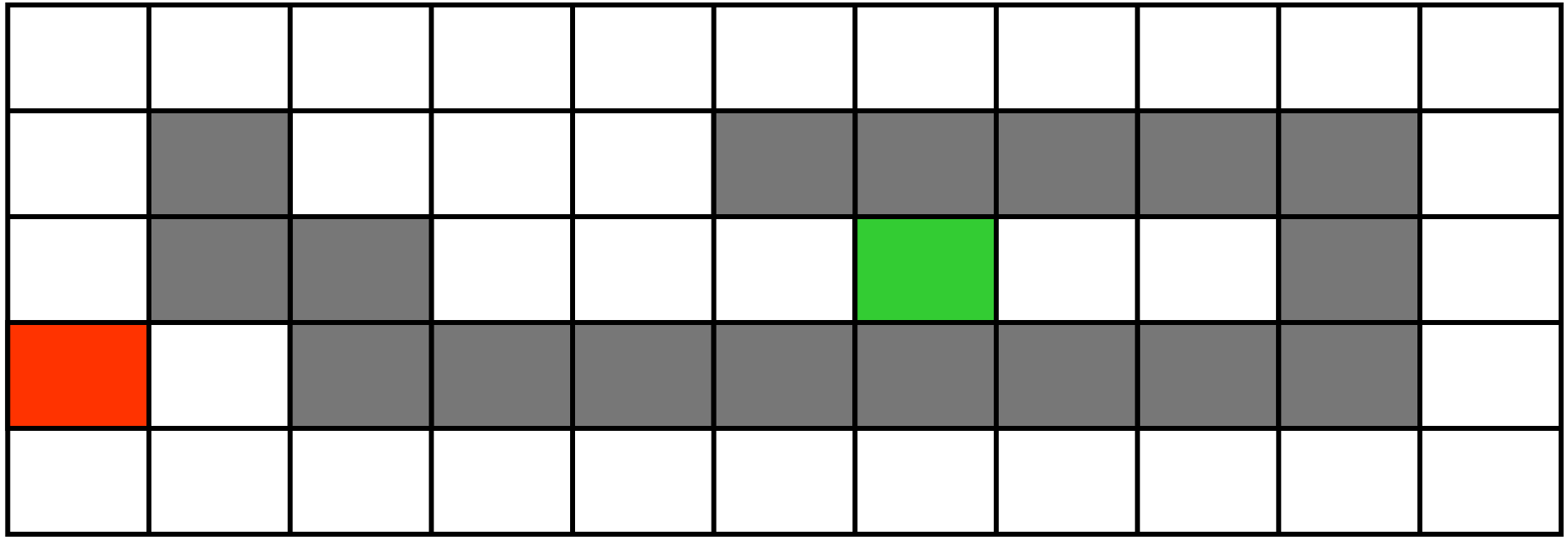
Tic Tac toe

- Heuristic function:
- No. of possible wins for you minus no of possible wins for opponents

Heuristic Search Techniques

- Best First Search
- A* search
- IDA*
- Local search algorithms
  - Hill Climbing
  - Simulated annealing
  - Genetic algorithms

Robot Navigation



Robot Navigation

f(N) = h(N), with h(N) = Manhattan distance to the goal
(not A*)

8	7	6	5	4	3	2	3	4	5	6
7		5	4	3						5
6			3	2	1	0	1	2		4
7	6									5
8	7	6	5	4	3	2	3	4	5	6

Robot Navigation

f(N) = h(N), with h(N) = Manhattan distance to the goal
(not A*)

8	7	6	5	4	3	2	3	4	5	6
7		5	4	3						5
6			3	2	1	0	1	2		4
7	6									5
8	7	6	5	4	3	2	3	4	5	6

Robot Navigation

f(N) = g(N)+h(N), with h(N) = Manhattan distance to goal (A*)

8+3	7+4	6+3	5+6	4+7	3+8	2+9	3+10	4	5	6
7+2		5+6	4+7	3+8						5
6+1			3	2+9	1+10	0+11	1	2		4
7+0	6+1									5
8+1	7+2	6+3	5+4	4+5	3+6	2+7	3+8	4	5	6

Local search algorithms

- In many optimization problems, the path to the goal is irrelevant; the goal state itself is the solution
- State space = set of "complete" configurations
- Find configuration satisfying constraints, e.g., n-queens
- In such cases, we can use local search algorithms
- keep a single "current" state, try to improve it

Local search algorithms

- Hill Climbing
  - Gradient Descent
  - Steepest Ascent
- Simulated annealing
- Genetic algorithms

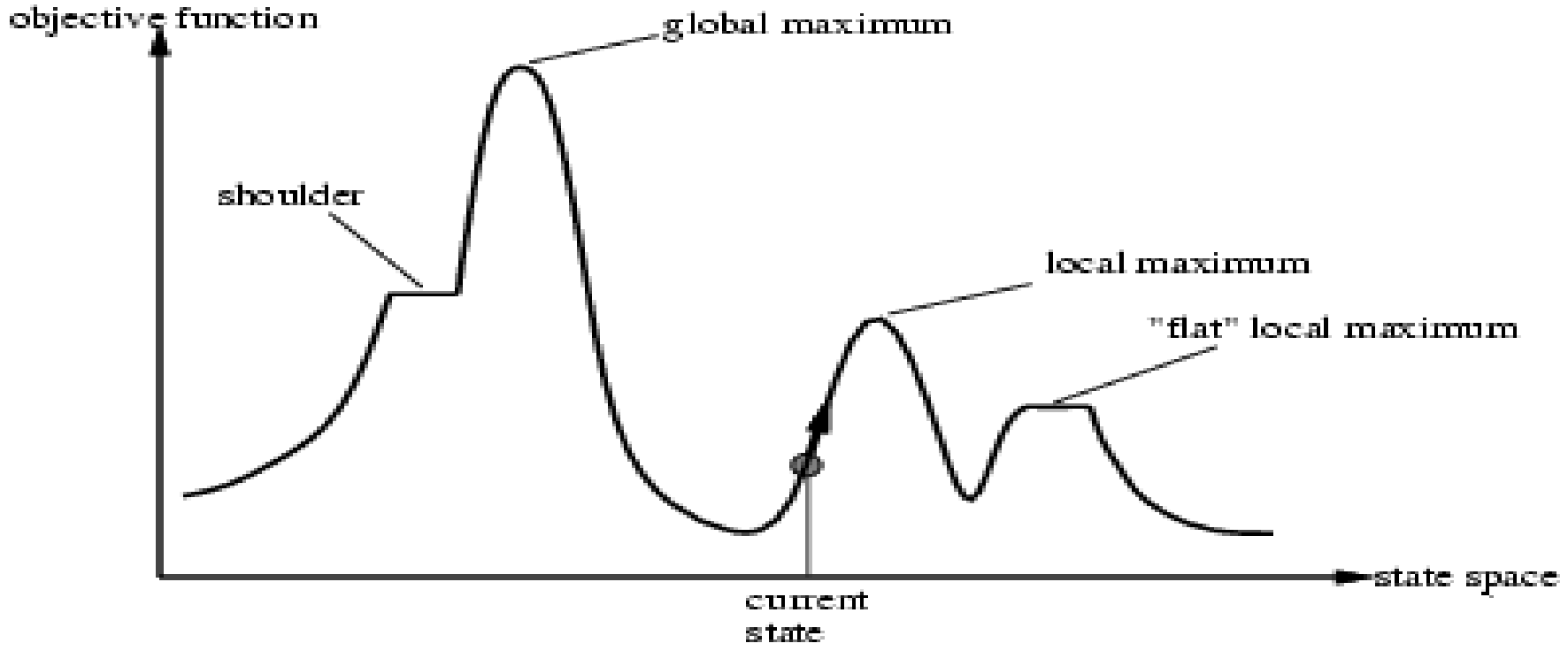
Hill-climbing search

- "Like climbing Everest in thick fog with amnesia"
-

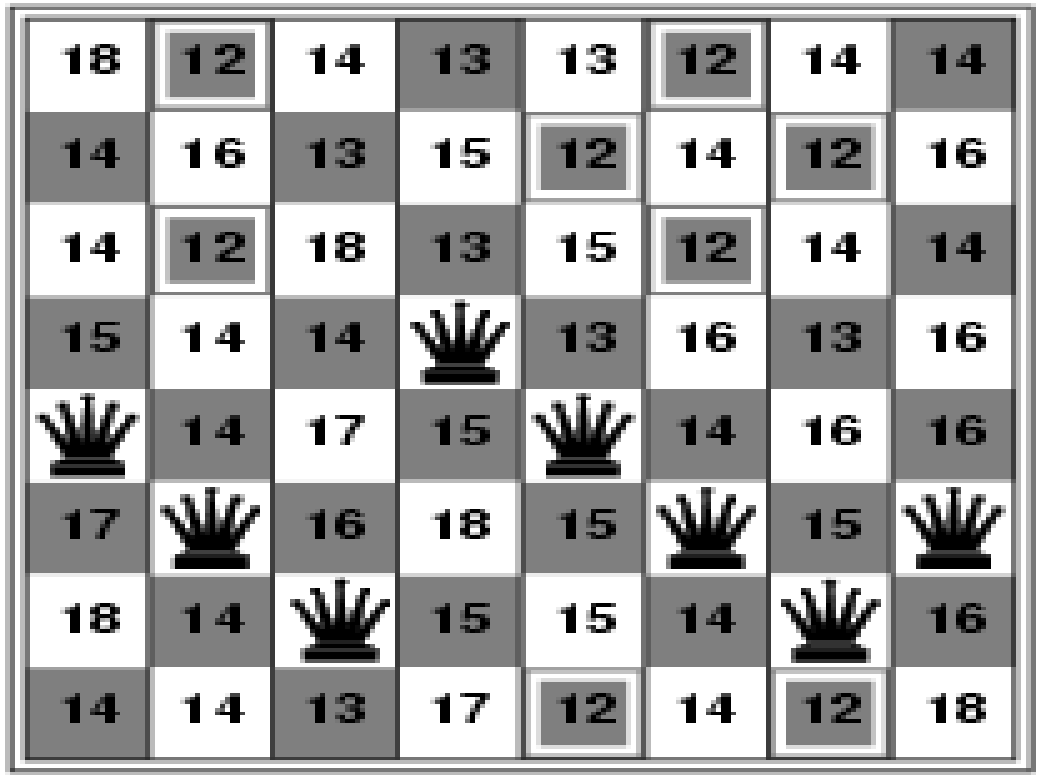
```
function HILL-CLIMBING(problem) returns a state that is a local maximum
   inputs: problem, a problem
   local variables: current, a node
                    neighbor, a node

   current ← MAKE-NODE(INITIAL-STATE[problem])
   loop do
      neighbor ← a highest-valued successor of current
      if VALUE[neighbor] ≤ VALUE[current] then return STATE[current]
      current ← neighbor
```

Hill-climbing search

- Problem: depending on initial state, can get stuck in local maxima
-

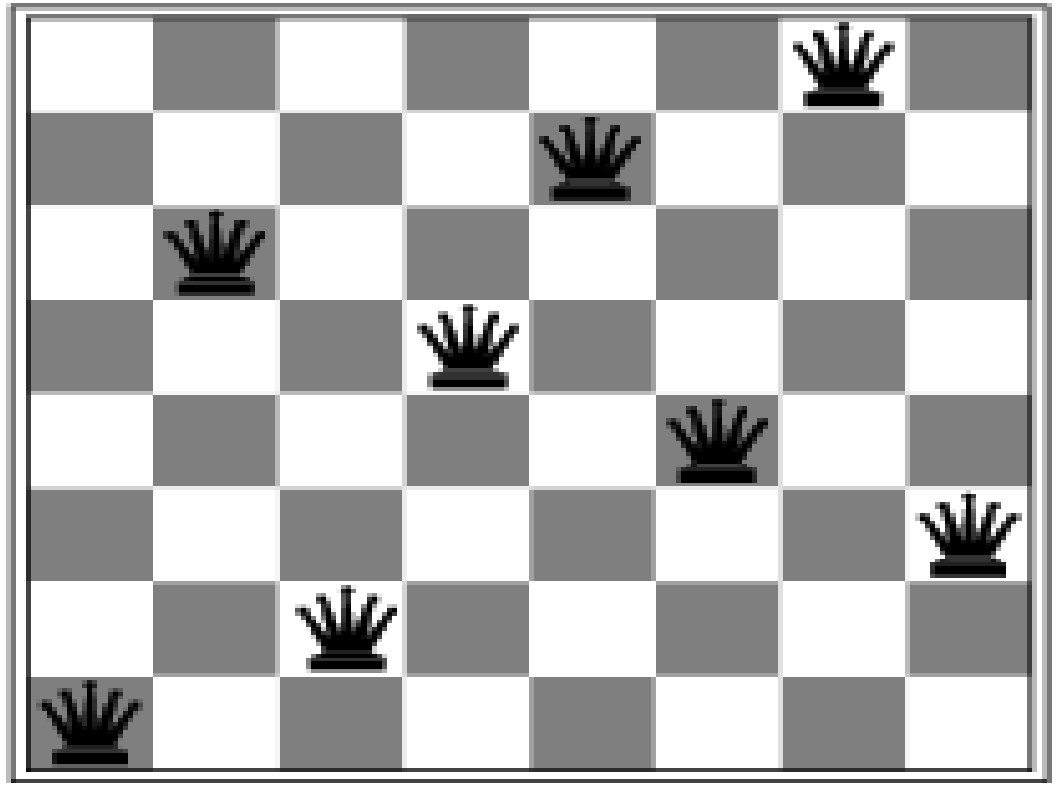


Hill-climbing search: 8-queens problem



- *h* = number of pairs of queens that are attacking each other, either directly or indirectly
- *h* = *17* for the above state

Hill-climbing search: 8-queens problem



- A local minimum with $h = 1$

Hill Climbing

- Use heuristic to move only to states that are better than the current state.
- Always move to better state when possible.
- Always climb uphill whenever you can, without looking around.

Hill Climbing

- Moves towards the goal
- Using heuristic, it finds which direction to move.
- It always find the node closer to the goal.
- HCSA is a loop that continuously moves in the direction of increasing values.

Hill Climbing Types

- Simple hill climbing:

  First closer node is chosen

- Steepest ascent/Gradient Descent hill climbing:

  All the successors are compared, closest to the solution is chosen.

Potential Problems with Simple Hill Climbing

- Will terminate when at local optimum.
- The order of application of operators can make a big difference.
- Can’t see past a single move in the state space.

Drawbacks

Local Maxima:

- It is a state which is better, all the other moves worsen the situation.

Plateau:

- An area of the search space where evaluation function is flat, thus requiring random walk.

Ridge:

- Where there are steep slopes and the search direction is not towards the top but towards the side.

Solutions to Hill climbing

- Local Maxima- Backtrack to some earlier node and try new direction
- Plateau: Make a big jump or apply the operators again and again though the situation continues
- Ridges: apply two or more operators before doing the test; will help in moving in several directions at once